SCHOOL ADMINISTARATION

CH.KARTHIK VARMA,
DEPT. NAME:SCHOOL OF COMPUTING AND DATA SCIENCE
SAI UNIVERSITY
CHENNAI,INDIA

P.SIVA KUMAR REDDY
DEPT. NAME:SCHOOL OF COMPUTING AND DATA SCIENCE
SAI UNIVERSITY
CHENNAI,INDIA

I.ABSTRACT

This project presents a School Administration System implemented in the C programming language using a Binary Search Tree (BST) data structure. The system efficiently manages student records by supporting operations such as insertion, deletion, searching, updating, and traversal. It incorporates dynamic memory allocation for handling strings and ensures memory safety through proper deallocation. Additional features include grade calculation, topper identification, filtering students based on marks, and counting total records. The use of BST enables efficient data organization and retrieval, making the system suitable for small to medium-scale academic institutions.

Keywords

Binary Search Tree, Student Management, Dynamic Memory Allocation, CRUD Operations, C Programming.

II.INTRODUCTION

Efficient management of student data is essential in educational institutions. Traditional methods of maintaining records manually are prone to errors and inefficiencies. This project introduces a digital solution using the C programming language, leveraging the Binary Search Tree (BST) data structure to store and manage student records.The BST allows for efficient insertion, deletion, and searching operations with an average time complexity of O(log n), making it suitable for structured data storage. This system provides a simple and effective console-based application for managing school records.

III.OBJECTIVES

The main objective of this project is to design and implement a Student Management System using a Binary Search Tree (BST) in the C programming language, demonstrating how data structures can be applied to manage student records efficiently. The system focuses on storing student details such as roll number, name, and marks in a structured manner that allows fast insertion, deletion, and searching operations by utilizing the properties of BST.The project also aims to perform complete CRUD operations—creating new student records, reading and displaying existing records, updating student information, and deleting records when required—while maintaining the integrity of the tree structure. It emphasizes the use of dynamic memory allocation in C through functions like malloc() and free(), ensuring proper memory usage and management during runtime.

IV.SCOPE PROJECT

This project is designed primarily for academic learning and small-scale applications, providing a practical understanding of how data structures like Binary Search Trees (BST) can be used to manage student records efficiently. It can be effectively used in schools or educational environments for maintaining basic student information such as roll numbers, names, and marks, while also serving as a hands-on tool for students to learn concepts like dynamic memory allocation, pointers, and tree operations in the C programming language,The project helps in visualizing how data is organized, inserted, searched, and deleted within a BST, making it useful for strengthening fundamental programming and problem-solving skills. It is especially beneficial for beginners who are learning about data structures, as it integrates theoretical knowledge with real-time implementation through a menu-driven console interface.

V.LITERATURE REVIEW

Binary Search Trees are widely used for hierarchical data storage due to their efficiency in search, insertion, and deletion operations, especially when the tree remains balanced. Standard data structure literature highlights BST as a fundamental concept for organizing and managing data in a structured and logical manner. This project leverages BST to store student records in a way that allows faster access compared to linear data structures.While modern applications typically rely on databases and advanced data management systems, this project focuses on in-memory data handling using the C programming language. This approach helps in understanding how data is stored and manipulated at a low level without the abstraction provided by database systems. It gives clear insight into how memory is allocated, accessed, and managed during program execution.

VI.METHODOLOGY

The system is developed using a structured and step-by-step approach to ensure clarity, efficiency, and maintainability. Initially, a student structure is defined to store essential details such as roll number, name, and marks, which forms the foundation of the entire system. Based on this structure, core BST operations such as insertion, searching, and deletion are implemented, ensuring that student records are organized according to BST properties for efficient access. These operations are designed using modular functions, making the code easier to understand, test, and

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

reuse. Special attention is given to maintaining the correctness of the tree during insertion and deletion, including handling all possible cases.In addition to the basic operations, several utility functions are developed to enhance the functionality of the system. These include grade calculation based on marks and different traversal methods for displaying student records in meaningful ways. Advanced features such as identifying the topper by comparing student marks and filtering students based on specific criteria are also incorporated to make the system more practical. Memory management is handled carefully using dynamic allocation and deallocation functions like malloc() and free(), ensuring efficient use of memory and preventing leaks. Overall, the system is designed to balance functionality, efficiency, and simplicity while demonstrating key concepts of data structures in C.

VII.TECHNOLOGY USED

The project is implemented using the C programming language, which provides low-level control over memory and efficient execution, making it ideal for understanding core data structure concepts. The program can be compiled and executed using standard compilers such as GCC or Turbo C, ensuring compatibility across different development environments. C is chosen specifically because it requires explicit handling of memory and pointers, which helps in gaining a deeper understanding of how data is stored and manipulated internally.The main concepts used in this project include Binary Search Trees for organizing student data efficiently, along with dynamic memory allocation using functions like malloc() and free() to manage memory at runtime. Recursion plays a key role in implementing tree operations such as insertion, traversal, searching, and deletion. Structures are used to define student records, while pointers are essential for linking nodes and navigating the tree. Together, these concepts provide a comprehensive foundation in both data structures and system-level programming, making the project highly valuable for learning and practical implementation.

VIII.ARCHITECTURE / SYSTEM DESIGN

The system is based on a Binary Search Tree (BST) structure in which each node represents a student record. Every node stores essential details such as a unique student ID, name, age, and marks, along with pointers to its left and right child nodes. The BST property is maintained throughout the system, where the left subtree contains students with IDs less than the root node, and the right subtree contains students with IDs greater than the root node. This organized structure enables efficient searching, insertion, and deletion operations, making it suitable for managing student data dynamically.The overall system is divided into three main modules: input, processing, and output. The input module is responsible for collecting student data from the user, ensuring proper entry of details. The processing module handles all core operations such as insertion into the BST, searching for specific records, deleting nodes, calculating grades, identifying the topper, and filtering students based on marks. The output module is designed to display the results in a clear and structured format using various traversal techniques, allowing users to view student information effectively. This modular design improves readability, maintainability, and scalability of the system.

IX.IMPLEMENTATION DETAILS

The implementation includes several key components that work together to build a complete and efficient student management system. A structure is first defined to represent student data, including fields such as student ID, name, age, and marks. Based on this structure, the insert function is implemented to add new student records into the Binary Search Tree while maintaining its ordering properties. The search function enables quick retrieval of student details using their unique ID, and the delete function is designed to remove records while correctly handling all cases, including deletion of leaf nodes, nodes with one child, and nodes with two children. Additionally, an update function is provided to modify existing student information without disrupting the structure of the tree.To support data presentation and additional features, inorder traversal is used to display student records in sorted order based on their IDs. A grade calculation function is also included to automatically assign grades according to student marks, making the system more practical and informative. Beyond these, the implementation may include other traversal methods like preorder and postorder for different viewing perspectives, as well as functions to identify the topper and filter students based on specific criteria. Memory management is handled carefully using malloc() for dynamic allocation and free() for deallocation, ensuring efficient use of memory and preventing memory leaks. Overall, the implementation combines core data structure operations with practical functionalities to create a well-rounded application.

X.RESULTS / OUTPUT

```
========================================
 SCHOOL ADMINISTRATION SYSTEM
========================================
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
6. Count Students
7. Find Topper
8. Students Above Marks
0. Exit
========================================
Enter your choice: 1
Enter Student ID: 1296
Enter Name: Rahul
Enter Age: 18
Enter Marks: 97
```

XI.CONCLUSION

This project successfully demonstrates the practical use of Binary Search Trees (BST) for managing student records in an efficient and organized manner. It highlights how operations such as insertion, searching, and deletion can be performed effectively using BST properties, resulting in improved performance compared to linear data structures.

The project also emphasizes the importance of dynamic memory management in C, showing how memory can be allocated and deallocated at runtime using functions like malloc() and free(). Through this, it provides a clear understanding of how data is handled internally, making the concepts of pointers, recursion, and structures more concrete and easier to grasp.In addition, the system proves to be suitable for small-scale applications where a simple and efficient method of managing data is required. It serves as a strong foundation for students to build their knowledge in data structures and algorithms, bridging the gap between theoretical concepts and real-world implementation. The project can be further extended by incorporating advanced features such as file handling for data persistence, graphical user interfaces for improved usability, or self-balancing trees to enhance performance. Overall, it acts as a stepping stone toward developing more complex and scalable software systems.

XII.FUTURE WORK

Future improvements to this project can significantly enhance its usability, performance, and scalability. One important enhancement would be the addition of a graphical user interface (GUI), which would make the system more user-friendly and visually interactive compared to the current console-based approach. Implementing file handling is another key improvement, allowing the system to store student data permanently so that records are not lost when the program terminates. This would make the application more practical for real-world use. Additionally, replacing the standard Binary Search Tree with self-balancing trees such as AVL Trees or Red-Black Trees can improve performance by ensuring that the tree remains balanced, thereby maintaining efficient operation times even with larger datasets.Further advancements can include integrating database systems like MySQL or SQLite to handle large-scale data more effectively and support multi-user environments. Adding user authentication and security features, such as login systems and role-based access control, would help protect sensitive student information and restrict unauthorized access. The system can also be extended with features like data validation, error handling, report generation, and analytics for better insights into student performance. Moreover, transitioning the application into a web-based or mobile-based platform could increase accessibility and usability. These improvements would transform the project from a basic academic implementation into a more robust and real-world applicable system.

XIII.REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

[2] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall, 1988. https://openlibrary.org/books/OL24604598M/The_C_Programming_Language